

Ventas-Backend

Guía técnica completa y preparación de entrevista Spring Boot
(nivel junior a senior)

Spring Boot 4.0.5

Java 21

PostgreSQL

JWT + Spring Security

JPA / Hibernate

Docker

130 tests

Proyecto: **ERP de Ventas** — microservicio backend (POS, inventario, compras, reportes)

Repositorio: `Ventas-Backend` (Maven, `com.erikjarquin:ventas`)

Documento generado: 18 de septiembre de 2026

Audiencia: quien estudia el proyecto y quien se prepara para una entrevista de Spring Boot

Indice

1. Que es el proyecto y alcance
2. De Spring Initializr al estado actual (cronologia)
3. Stack tecnologico y por que
4. El `pom.xml` explicado
5. Arquitectura por capas y estructura de paquetes
6. Modelo de dominio y relacion con la base de datos
7. Configuracion: `application.yaml` vs `application-local.yaml`
8. `.gitignore`, `.dockerignore` y `Dockerfile`
9. Seguridad: Spring Security + JWT + permisos
10. Controllers: la puerta de entrada HTTP
11. Services: logica de negocio y transacciones
12. Repositories: como se construyen las consultas
13. DTOs y Mappers
14. Excepciones y `GlobalExceptionHandler`
15. Clases de `config/` y bootstraps
16. Jobs programados (`@Scheduled`)
17. Tests: tipos, ejemplos y como ejecutarlos
18. Comandos utiles (Maven wrapper y Docker)
19. Catalogo de endpoints por modulo
20. Flujos clave paso a paso
21. Decisiones de diseno y buenas practicas
22. Deuda tecnica y posibles mejoras
23. Preguntas tipicas de entrevista (con respuestas del proyecto)
24. Glosario rapido

1. Que es el proyecto y alcance

Ventas-Backend es la API REST de un ERP de punto de venta (POS). Es un microservicio construido con **Spring Boot** que expone endpoints bajo `/api/...` y que un frontend Angular (proyecto hermano **Ventas-Frontend**) consume. Resuelve el ciclo comercial completo:

- **Seguridad:** login con JWT, roles y permisos granulares, recuperacion de contrasena por correo.
- **Catalogo:** productos (con imagen), categorias, busqueda por nombre/SKU/codigo de barras, borrado logico.
- **Ventas (POS):** venta en efectivo o con tarjeta, descuento de stock, historial.
- **Pagos:** integracion con terminal de tarjeta (simulada o fisica), reintentos, reversas e idempotencia.
- **Caja:** apertura, cierre, corte y consulta del estado.
- **Compras:** proveedores y compras que suman stock y guardan el costo real.
- **Reportes:** tendencias, top productos, metodos de pago, categorias, stock bajo, margenes y resumen.

La API es *stateless*: cada peticion se autentica con un token JWT en la cabecera `Authorization: Bearer <token>`. La autorizacion fina se hace por **permiso** (no solo por rol) con `@PreAuthorize`.

2. De Spring Initializr al estado actual (cronologia)

El proyecto nace de [Spring Initializr](#) (start.spring.io) seleccionando Maven, Java 21 y los starters de `Spring Web`, `Spring Data JPA`, `Spring Security`, `Validation` y `PostgreSQL Driver`. Eso genera la estructura base: `pom.xml`, `mvnw / mvnw.cmd`, `src/main/java`, `src/main/resources`, `application.properties` (aqui se usa `application.yaml`) y una clase `@SpringBootApplication`.

Sobre esa base, el proyecto fue creciendo por sesiones. Resumen de hitos (tomado de `AGENTS.md`):

Fecha / hito	Que se hizo
Base	Auth JWT, usuarios, roles, permisos, productos, categorias, ventas y caja.
2026-09-09	<code>@EnableScheduling</code> (activa jobs), arreglo de repositorio, almacenamiento real de imagenes (<code>FileStorageService</code> + <code>WebConfig</code>) y URL completa de imagen en el DTO.
2026-09-12	Secretos externalizados a variables de entorno (preparacion Railway), CORS global, <code>GlobalExceptionHandler</code> uniforme, Dockerfile multi-etapa, tests sobre H2.
2026-09-12 (3)	Modulo de Reportes completo: consultas de agregacion en SQL/JPQL que devuelven DTOs.
2026-09-13	Frontend conectado, <code>PUT</code> de categorias y estados HTTP correctos (409/404).
2026-09-13 (2)	Modulo de Compras completo: proveedores, compras, stock y costo real, reporte de margenes.
2026-09-15	Correccion de integridad referencial: no borrar productos/categorias/roles con historico (409), imagenes huérfanas => 404.
2026-09-17	Eliminado <code>GET /ping</code> , flag de seed (<code>APP_SEED_BOOTSTRAPS</code>), fin del self-healing de permisos, tests de controllers.
2026-09-17 (3)	Borrado logico de productos ("dar de baja"/"reactivar") conservando historico.

Para la entrevista: poder contar la evolucion del proyecto (empezar simple y endurecerlo: secretos, tests, integridad, transacciones) demuestra criterio de ingenieria, no solo saber escribir CRUDs.

3. Stack tecnologico y por que

Tecnología	Uso en el proyecto	Por que
Java 21	Lenguaje	LTS, records, pattern matching, mejor rendimiento.
Spring Boot 4.0.5	Framework base	Auto-configuracion, servidor embebido, ecosistema de starters.
Spring Web (MVC)	API REST	Controladores anotados, <code>DispatcherServlet</code> , JSON con Jackson.
Spring Data JPA + Hibernate 7	Persistencia	Repositorios sin implementacion, JPQL, mapeo objeto-relacional.
PostgreSQL	Base de datos	Relacional robusto; en local <code>ventas_db</code> .
H2	BD de tests	En memoria: los tests corren sin levantar Postgres.
Spring Security + jjwt 0.11.5	Autenticacion JWT	Filtros, contexto de seguridad y tokens firmados HS256.
Bean Validation	Validar entrada	Anotaciones <code>@NotBlank</code> , <code>@Email</code> , <code>@Min</code> .
Spring Mail	Correo SMTP	Recuperacion de contrasena via Gmail.
Lombok	Menos codigo	<code>@Data</code> , <code>@RequiredArgsConstructor</code> , <code>@Slf4j</code> .
Docker multi-etapa	Despliegue	Imagen ligera (jre21) para Railway.
Maven (wrapper)	Build	<code>mvnw.cmd</code> no exige Maven instalado.

4. El `pom.xml` explicado

El `pom.xml` es el contrato de Maven: identidad del proyecto, dependencias y plugins. El proyecto hereda de `spring-boot-starter-parent`, que fija versiones compatibles entre librerías.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.0.5</version>
</parent>

<groupId>com.erikjarquin</groupId>
<artifactId>ventas</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
  <java.version>21</java.version>
</properties>
```

Dependencias principales

Artefacto	Rol
spring-boot-starter-web	MVC, Tomcat embebido, Jackson.
spring-boot-starter-data-jpa	JPA/Hibernate + repositorios Spring Data.
spring-boot-starter-security	Cadena de filtros, BCrypt, @PreAuthorize .
spring-boot-starter-validation	Bean Validation (Jakarta).
jjwt-api / jjwt-impl / jjwt-jackson	Generar/validar JWT (HS256) y serializar claims.
postgresql (runtime)	Driver JDBC de Postgres.
h2 (runtime)	BD en memoria usada por los tests.
spring-boot-starter-mail	Envio de correos de recuperacion.
lombok (optional)	Genera getters/setters/constructores/logs en compilacion.
spring-boot-starter-data-jpa-test, -validation-test, -webmvc-test, -security-test	Utilidades de test: @DataJpaTest, @WebMvcTest, @WithMockUser, Mockito.

```
<!-- Tests de controladores: @WebMvcTest/MockMvc/Mockito y @WithMockUser -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

El plugin `spring-boot-maven-plugin` empaqueta el jar ejecutable ("fat jar") que se copia en el Dockerfile. El scope `test` hace que esas dependencias no viajen al artefacto de produccion.

5. Arquitectura por capas y estructura de paquetes

El codigo sigue una separacion por responsabilidades (arquitectura en capas) bajo `com.erikjarquin.ventas` :

```
controller/      -> reciben HTTP, validan permisos, delegan al service. Devuelven DTOs.
service/         -> interfaces de negocio.
service/impl/    -> implementaciones (@Service), transacciones y reglas.
repository/      -> interfaces Spring Data JPA (@Query, derived queries, @EntityGraph).
model/entity/    -> entidades JPA (tablas).
model/dto/       -> objetos de transferencia (request/response). Agrupados por modulo.
model/enums/     -> PermissionName, PaymentMethod, PaymentStatus, ReportGroup.
mapper/          -> conversion entidad <-> DTO (estaticos o @Component).
config/          -> seguridad, CORS, JWT, WebConfig, bootstraps, @ConfigurationProperties.
exceptions/      -> excepciones de negocio + GlobalExceptionHandler.
jobs/            -> tareas programadas (@Scheduled).
```

El flujo tipico de una peticion:

Cliente HTTP

- > JwtFilter (autentica: valida token y carga permisos de BD)
- > DispatcherServlet enruta al @RestController
- > @PreAuthorize valida el permiso (403 si falta)
- > Controller llama al Service (DTO de entrada)
- > Service aplica reglas + usa Repository (entidades)
- > Repository ejecuta SQL/JPQL en PostgreSQL
- > Service mapea a DTO
- > Controller responde JSON (o lanza excepcion -> GlobalExceptionHandler)

Regla de oro: las entidades JPA NO salen al exterior; lo que viaja por HTTP son DTOs. Esto evita exponer el modelo interno, problemas de lazy-loading al serializar y acoplar la API al esquema de BD.

6. Modelo de dominio y relacion con la base de datos

Cada clase `@Entity` corresponde a una tabla. Hibernate (con `ddl-auto: update`) crea y actualiza el esquema a partir de las entidades en desarrollo. Tablas del sistema:

Entidad	Tabla	Descripcion
UserEntity	users	Usuarios (email unico, password BCrypt, active, token de reset).
RoleEntity	roles	Roles (ADMIN, CAJERO, ALMACENISTA...).
PermissionEntity	permissions	Catalogo de permisos (enum <code>PermissionName</code>).
(tabla intermedia)	role_permissions	Relacion N:M rol-permiso (<code>@ManyToMany</code>).
ProductEntity	products	Productos: precio, cost (costo real), stock, SKU/codigo unicos, active .
CategoryEntity	categories	Categorias de productos.
SaleEntity	sales	Cabecera de venta: total, metodo, estado de pago, caja.
SaleDetailEntity	sale_details	Renglones: producto, cantidad, precio unitario, subtotal.
PaymentEntity	payments	Pago con tarjeta: transaccion, estado, reintentos, reversa.
CashRegisterEntity	cash_registers	Caja: apertura/cierre, montos, active .
ProviderEntity	providers	Proveedores: RFC unico, telefono, email.
PurchaseEntity	purchases	Cabecera de compra: proveedor, fecha, total.
PurchaseDetailEntity	purchase_details	Renglones: producto, cantidad, costo unitario, subtotal.

Relaciones principales

```
Role 1---N User      (un rol tiene muchos usuarios; User.role es @ManyToOne LAZY)
Role N---N Permission (role_permissions; alimenta las autoridades del JWT)
Category 1---N Product (Product.category @ManyToOne LAZY)
Sale 1---N SaleDetail (cascade = ALL, orphanRemoval = true)
Sale N---1 CashRegister (la venta pertenece a la caja abierta)
Sale 1---1 Payment    (solo si la venta fue con tarjeta)
Provider 1---N Purchase (proveedor de la compra)
Purchase 1---N PurchaseDetail (cascade = ALL, orphanRemoval = true)
PurchaseDetail N---1 Product
SaleDetail N---1 Product
```

Ejemplos reales de mapeo (lado de la venta):

```

@Entity
@Table(name = "sales")
public class SaleEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING) // guarda el TEXTO del enum, no un ordinal
    private PaymentMethod paymentMethod;

    @OneToMany(mappedBy = "sale", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<SaleDetailEntity> details;

    @ManyToOne @JoinColumn(name = "cash_register_id")
    private CashRegisterEntity cashRegister;
}

```

Detalle de entrevista: `@Enumerated(STRING)` guarda el nombre legible (`APPROVED`) en vez del índice numérico. Es más seguro ante reordenamientos del enum, y es la razón por la que existe un CHECK en Postgres con los nombres válidos (ver sección 8, restricción `permissions_name_check`).

Borrado lógico vs físico. Productos y usuarios usan un flag `active` : "eliminar" en realidad desactiva para no perder el historico (ventas/compras referencian por FK). El borrado físico solo se permite si no hay historico; si lo hay, se responde `409 Conflict` .

7. Configuración: `application.yaml` VS `application-local.yaml`

Hay dos archivos con roles distintos:

- `src/main/resources/application.yaml` : **se sube a Git**. Define la estructura de configuración y los *placeholders* `${VARIABLE:default}` , pero **sin secretos**.
- `src/main/resources/application-local.yaml` : **NO se sube** (esta en `.gitignore`). Contiene los valores reales de desarrollo (usuario/clave de BD, JWT secret, etc.). Se carga por el perfil `local` , activo por defecto.

```
# application.yaml (fragmento, sin secretos)
spring:
  profiles:
    default: local          # en tu maquina carga application-local.yaml
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:ventas_db}
    username: ${DB_USER}    # SIN default: si falta, la app NO arranca
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: ${JPA_DDL_AUTO:update}
      show-sql: ${SHOW_SQL:true}
  mail:
    username: ${MAIL_USERNAME}
    password: ${MAIL_PASSWORD}

app:
  frontend-url: ${FRONTEND_URL:http://localhost:4200}
  seed-bootstraps: ${APP_SEED_BOOTSTRAPS:true}
  upload-dir: ${UPLOAD_DIR:./uploads}
  upload-url: ${UPLOAD_URL:http://localhost:8081/api/uploads}

server:
  port: ${PORT:8081}

payment:
  terminal:
    type: ${PAYMENT_TERMINAL_TYPE:SIMULATED}
    merchant-id: ${PAYMENT_MERCHANT_ID}
```

Orden de precedencia para resolver \${...}

1. Variables de entorno del sistema / de la plataforma (Railway). **Ganan.**
2. `application-local.yaml` (perfil `local` activo).
3. `application.yaml` (valores por defecto tras los dos puntos: `${VAR:default}`).

Fail-fast intencional: variables como `${DB_USER}` o `${JWT_SECRET}` no tienen default. Si no se resuelven, la aplicacion falla al arrancar en vez de correr con valores vacios. Es una decision de seguridad.

Perfiles de Spring

Un *perfil* agrupa configuracion por entorno. Al arrancar se activa `local` por defecto; en produccion, el archivo `application-local.yaml` no existe y los valores llegan por variables de entorno. Se puede forzar con `--spring.profiles.active=prod` o `SPRING_PROFILES_ACTIVE=prod`.

8. `.gitignore`, `.dockerignore` y `Dockerfile`

`.gitignore` (que NO se versiona)

```
target/                # compilados
.env / .env.*          # variables de entorno
application-local.yaml  # credenciales de desarrollo
uploads/               # imagenes subidas (se regeneran)
*.jks *.p12 *.cer *.crt *.key *.pem # certificados / keystores
*.log                  # logs (pueden contener SQL/datos)
.vscode/ .idea/ *.iml  # basura de IDE
```

Nota: `AGENTS.md` (el historial de decisiones del proyecto) si se versiona a proposito para compartir contexto entre maquinas.

.dockerignore (que NO entra al contexto de build de Docker)

Cumple doble proposito: **seguridad** (no copiar `application-local.yaml`, `.env`, keystores) y **rendimiento** (no mandar `target/`, `.git/`, IDE). Una imagen no debe llevar secretos de desarrollo dentro.

Dockerfile multi-etapa

```
# Etapa 1: compilar (imagen con Maven + JDK 21)
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn -q dependency:go-offline      # cachea dependencias en una capa
COPY src ./src
RUN mvn clean package -DskipTests    # los tests necesitan BD; se omiten en build

# Etapa 2: ejecutar (imagen final ligera: solo JRE)
FROM eclipse-temurin:21-jre
WORKDIR /app
RUN addgroup --system appgroup \
    && adduser --system --ingroup appgroup --uid 1001 appuser \
    && mkdir -p /app/uploads \
    && chown -R appuser:appgroup /app
USER appuser                      # NO ejecutar como root
COPY --from=build /build/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Por que multi-etapa: la imagen final no incluye Maven ni el codigo fuente, solo el JRE y el jar. Resultado: imagen mas pequena y menor superficie de ataque. El build copia primero el `pom.xml` para aprovechar la cache de capas (si no cambian dependencias, no se re-descargan).

Usuario no-root y volumen: en Railway se monta un volumen persistente en `/app/uploads` (mismo UID 1001) para que las imagenes de productos sobrevivan a los redeloys.

9. Seguridad: Spring Security + JWT + permisos

La seguridad tiene tres piezas clave: **como se emite el token** (`JwtUtil`), **como se valida en cada request** (`JwtFilter`) y **como se autoriza** (`SecurityConfig` + `@PreAuthorize` + `SecurityAuthorityMapper`).

9.1 `JwtUtil`: emitir y validar el token

```
@Component
public class JwtUtil {
    @Value("${JWT_SECRET}") // nunca hardcodeado: viene de env/local
    private String secretKey;

    public String generateToken(UserEntity user) {
        return Jwts.builder()
            .setSubject(user.getEmail()) // el "subject" es el email
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 1000L*60*60*5)) // 5 h
            .signWith(Keys.hmacShaKeyFor(secretKey.getBytes(UTF_8)), SignatureAlgorithm.HS256)
            .compact();
    }

    public boolean isValidToken(String token) {
        try { getClaims(token); return true; }
        catch (Exception e) { return false; } // firma mala/expirado => false
    }
}
```

HS256 es firma simétrica (un solo secreto para firmar y validar). El secreto debe tener al menos 32 bytes. Si se filtra, cualquiera puede forjar un token de ADMIN: por eso vive en variable de entorno.

9.2 `JwtFilter`: autenticar cada request

```
@Component
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String authHeader = req.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(req, res); return; // sin token: sigue (401/403 luego)
        }
        try {
            String token = authHeader.substring(7);
            String email = jwtUtil.extractEmail(token);
            if (jwtUtil.isValidToken(token) && SecurityContextHolder.getContext().getAuthentication() == null) {
                // Carga usuario + rol + permisos de la BD EN CADA REQUEST
                UserEntity user = userRepository.findByEmailWithRoleAndPermissions(email).orElse(null);
                if (user != null) {
                    var auth = new UsernamePasswordAuthenticationToken(
                        user, null, authorityMapper.mapAuthorities(user));
                    SecurityContextHolder.getContext().setAuthentication(auth);
                }
            }
        } catch (JwtException | IllegalArgumentException e) {
            SecurityContextHolder.clearContext(); // token invalido: no romper con 500
        }
        chain.doFilter(req, res);
    }
}
```

Gran decision de diseño: el JWT solo lleva el email. Los permisos se recargan de la BD en cada request. Así, si a un rol se le quita un permiso, el cambio aplica *de inmediato* sin esperar a que expire el token (5 h). Es un balance entre rendimiento y frescura de permisos.

9.3 SecurityAuthorityMapper : rol + permisos a autoridades

```
public List<GrantedAuthority> mapAuthorities(UserEntity user) {
    List<GrantedAuthority> authorities = new ArrayList<>();
    authorities.add(new SimpleGrantedAuthority("ROLE_" + user.getRole().getName()));
    for (PermissionEntity p : user.getRole().getPermissions()) {
        authorities.add(new SimpleGrantedAuthority(p.getName().name()));
    }
    return authorities;
}
```

Esto habilita tanto `hasRole("ADMIN")` (autoridad `ROLE_ADMIN`) como `hasAuthority("CREAR_VENTAS")` (una autoridad por permiso).

9.4 SecurityConfig : cadena de filtros y CORS

```
@Configuration
@EnableMethodSecurity // habilita @PreAuthorize
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // API stateless con JWT: no hay cookies
            .cors(Customizer.withDefaults())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "/*").permitAll() // preflight CORS
                .requestMatchers("/api/auth/*").permitAll()
                .requestMatchers("/api/uploads/*").permitAll() // imagenes publicas
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

El `CorsConfigurationSource` lee `${CORS_ALLOWED_ORIGINS}` (lista separada por comas) en vez de usar `@CrossOrigin` en cada controller.

9.5 Permisos, roles y bootstraps

El enum `PermissionName` define los 34 permisos del sistema (por modulo): productos, categorias, usuarios, roles, ventas, caja, compras, proveedores, reportes y pagos. Cada endpoint se protege así:

```
@PreAuthorize("hasAuthority('VER_PRODUCTOS')")
@GetMapping
public List<ProductDto> getProducts(...) { ... }
```

Cuatro `CommandLineRunner` siembran la BD al arrancar (orden garantizado con `@Order`):

Orden	Bootstrap	Responsabilidad
1	RoleBootstrap	Crea los roles base (ADMIN, CAJERO, ALMACENISTA) si la tabla esta vacia.
2	AdminBootstrap	Crea el usuario admin inicial si su email no existe.
3	PermissionBootstrap	Inserta los permisos del enum que falten (solo agrega, nunca borra).
4	RolePermissionBootstrap	Asigna permisos a los roles base <i>solo la primera vez</i> (no self-healing).

```

@Component
@Order(3)
public class PermissionBootstrap implements CommandLineRunner {
    public void run(String... args) {
        if (!seedEnabled) return; // respeta APP_SEED_BOOTSTRAPS
        for (PermissionName name : PermissionName.values()) {
            if (permissionRepository.findByName(name).isEmpty())
                permissionRepository.save(new PermissionEntity(null, name));
        }
    }
}

```

Leccion aprendida (documentada en AGENTS.md): un bootstrap "self-healing" que reasignaba permisos en cada arranque "resucitaba" roles y permisos borrados a mano. Ahora los seeds solo actuan la primera vez y respetan `APP_SEED_BOOTSTRAPS`. Es un ejemplo clasico de *efecto no deseado* de una tarea de arranque.

10. Controllers: la puerta de entrada HTTP

Un `@RestController` mapea rutas a metodos Java. **No** llevan logica de negocio: validan el permiso, reciben parametros/body y delegan en el service.

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service; // inyeccion por constructor

    @PreAuthorize("hasAuthority('VER_PRODUCTOS')")
    @GetMapping
    public List<ProductDto> getProducts(@RequestParam(required = false) String category) {
        if (category != null) return service.getByCategory(category);
        return service.getAll();
    }

    @PreAuthorize("hasAuthority('CREAR_PRODUCTOS')")
    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ProductDto saveProduct(@RequestParam String name, ...,
                                  @RequestParam(required=false) MultipartFile image) {
        return service.save(name, price, stock, categoryId, sku, barcode, image);
    }

    @PreAuthorize("hasAuthority('DESACTIVAR_PRODUCTOS')")
    @PatchMapping("/{id}/deactivate")
    public ProductDto deactivateProduct(@PathVariable Long id) {
        return service.deactivate(id);
    }
}
```

Anotaciones clave:

Anotacion	Significado
<code>@RestController</code>	Devuelve el objeto como JSON (equivale a <code>@Controller</code> + <code>@ResponseBody</code>).
<code>@RequestMapping</code>	Prefijo comun de rutas del controller.
<code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PutMapping</code> / <code>@DeleteMapping</code> / <code>@PatchMapping</code>	Verbo HTTP del endpoint.
<code>@PathVariable</code>	Valor embebido en la URL (<code>/products/{id}</code>).
<code>@RequestParam</code>	Parametro de query o de formulario.
<code>@RequestBody</code>	JSON del cuerpo convertido a DTO.
<code>ResponseEntity<T></code>	Control total del codigo HTTP (201, 204...).

Convencion REST usada: GET para leer, POST para crear, PUT para reemplazar/editar, PATCH para cambios parciales (`/active` , `/deactivate`) y DELETE para eliminar. Codigos: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401/403 seguridad, 404 no encontrado, 409 conflicto, 500 interno.

11. Services: logica de negocio y transacciones

Se separa la **interfaz** (`service/XxxService.java`) de la **implementacion** (`service/impl/XxxImpl.java`). El controller depende de la interfaz: facilita testear con mocks y cambiar la implementacion sin tocar el controller.

Ejemplo: crear una compra (transaccion)

```
@Override
@Transactional // TODO ocurre en UNA transaccion (todo o nada)
public PurchaseDTO create(PurchaseRequest request) {
    validateRequest(request);
    ProviderEntity provider = providerRepository.findById(request.getProviderId())
        .orElseThrow(() -> new PurchaseException("El proveedor indicado no existe", HttpStatus.CONFLICT));

    PurchaseEntity purchase = new PurchaseEntity();
    purchase.setPurchaseDate(...);
    purchase.setProvider(provider);

    BigDecimal total = BigDecimal.ZERO;
    for (PurchaseItemRequest item : request.getItems()) {
        ProductEntity product = productRepository.findById(item.getProductId()).orElseThrow(...);
        BigDecimal unitCost = item.getUnitCost() != null ? item.getUnitCost() : BigDecimal.ZERO;

        PurchaseDetailEntity detail = new PurchaseDetailEntity();
        detail.setProduct(product);
        detail.setQuantity(item.getQuantity());
        detail.setUnitCost(unitCost);
        detail.setSubtotal(unitCost.multiply(BigDecimal.valueOf(item.getQuantity())).setScale(2, HALF_UP));
        total = total.add(detail.getSubtotal());

        product.setStock(product.getStock() + item.getQuantity()); // suma stock
        product.setCost(unitCost); // guarda costo real
        productRepository.save(product);
    }
    purchase.setTotal(total.setScale(2, HALF_UP));
    return PurchaseMapper.toDto(purchaseRepository.save(purchase));
}
```

ACID y rollback: anotar el metodo con `@Transactional` hace que, si algo falla (por ejemplo un producto inexistente a mitad del bucle), toda la operacion se revierta. Sin transaccion quedarian productos con stock sumado y la compra sin guardar (inconsistencia). `BigDecimal` se usa para dinero (evita errores de redondeo de `double`) y `HALF_UP` define el redondeo a 2 decimales.

Ejemplo: venta con tarjeta (flujo transaccional complejo)

```
@Override
@Transactional(rollbackFor = Exception.class)
public SaleResponse processSale(SaleRequest request) {
    validateRequest(request);
    CashRegisterEntity cash = getActiveCashRegister(); // debe haber caja abierta

    SaleEntity sale = createBaseSale(request, cash);
    sale.setPaymentStatus(PaymentStatus.PENDING);
    SaleEntity savedSale = saleRepository.save(sale); // guarda para obtener ID

    ProcessedProducts processed = processProductsInMemory(request.getItems(), sale);
    savedSale.setDetails(processed.getDetails());
    savedSale.setTotal(processed.getTotal());

    if (request.getPaymentMethod() == PaymentMethod.CASH) {
        processCashPayment(request, savedSale); // valida el cambio
    } else {
        paymentResponse = processCardPaymentWithResponse(request, savedSale); // delega en la terminal
    }

    savedSale.setPaymentStatus(PaymentStatus.APPROVED); // solo si el pago paso
    for (SaleDetailEntity detail : savedSale.getDetails()) {
        ProductEntity p = detail.getProduct();
        p.setStock(p.getStock() - detail.getQuantity()); // descuenta stock
        productRepository.save(p);
    }
    return mapper.toResponse(saleRepository.save(savedSale));
}
```

Patron importante: el stock se descuenta SOLO cuando el pago fue aprobado, y todo el metodo es transaccional. Si el cobro falla, el `rollback` deshace la venta PENDING y no descuenta stock.

12. Repositories: como se construyen las consultas

Un `JpaRepository<T, ID>` aporta CRUD gratis y permite declarar consultas de tres formas:

a) Derived queries (por nombre de metodo)

```
List<ProductEntity> findByActiveTrue();
List<ProductEntity> findByCategory_NameAndActiveTrue(String name);
Optional<ProductEntity> findByBarcode(String barcode);
long countByRole_Id(Long roleId);
```

Spring Data traduce el nombre a SQL. `Category_Name` navega la relacion (`product.category.name`).

b) JPQL con `@Query` (consulta de objetos, no de tablas)

```
@Query("""
    SELECT p FROM ProductEntity p
    WHERE p.active = true
    AND ( LOWER(p.name)      LIKE LOWER(CONCAT('%', :q, '%'))
        OR LOWER(p.sku)     LIKE LOWER(CONCAT('%', :q, '%'))
        OR LOWER(p.barcode) LIKE LOWER(CONCAT('%', :q, '%')) )
""")
List<ProductEntity> search(@Param("q") String q);
```

c) Consultas de agregacion (modulo de reportes)

```
@Query("""
    SELECT CAST(s.saleDate AS date), SUM(s.total), COUNT(s)
    FROM SaleEntity s
    WHERE s.paymentStatus = :status AND s.saleDate >= :from AND s.saleDate < :to
    GROUP BY CAST(s.saleDate AS date)
    ORDER BY CAST(s.saleDate AS date)
""")
List<Object[]> sumSalesByDay(@Param("from") LocalDateTime from,
                             @Param("to") LocalDateTime to,
                             @Param("status") PaymentStatus status);
```

Agregar en SQL, no en Java: SUM/COUNT/GROUP BY los hace la base de datos (mucho mas eficiente que traer miles de filas y sumarlas en un `for`). El backend devuelve DTOs y el frontend solo grafica. El filtro `paymentStatus = APPROVED` evita contar ventas rechazadas o reversadas.

El problema N+1 y `@EntityGraph`

Si al listar 100 compras se consulta el proveedor y el producto de cada una por separado, se hacen ~200 consultas (el clasico *N+1*). La solucion es precargar con `@EntityGraph` o `JOIN FETCH`:

```

@EntityGraph(attributePaths = {"provider", "details.product"})
List<PurchaseEntity> findAllByOrderByPurchaseDateDesc();

// UserRepository: JOIN FETCH del rol y sus permisos en UNA consulta
@Query("""
    SELECT DISTINCT u FROM UserEntity u
        JOIN FETCH u.role r
        JOIN FETCH r.permissions
    WHERE u.email = :email
""")
Optional<UserEntity> findByEmailWithRoleAndPermissions(@Param("email") String email);

```

Otro caso resuelto con cuidado: calcular `hasHistory` de cada producto. En vez de un `exists` por producto, se traen todos los IDs con historico en 2 consultas y se consulta en memoria:

```

@Query("SELECT DISTINCT d.product.id FROM SaleDetailEntity d WHERE d.product IS NOT NULL")
List<Long> findProductIdsWithSales();

// en ProductImpl: 2 consultas + HashSet, en vez de N exists
Set<Long> withHistory = new HashSet<>(repository.findProductIdsWithSales());
withHistory.addAll(repository.findProductIdsWithPurchases());

```

13. DTOs y Mappers

Un **DTO** (Data Transfer Object) es un objeto plano que define que datos entran o salen de la API. Se organizan en `model/dto/<Modulo>/` (por ejemplo `ProductDto`, `CreateUserRequest`, `PurchaseRequest`, `MarginDTO`).

Un **Mapper** convierte entidad `<->` DTO. Los hay estaticos (utilidades) y `@Component` (cuando necesitan dependencias, como `ProductMapper` que arma la URL de imagen):

```

@Component
public class ProductMapper {
    @Value("${app.upload-url}") String uploadUrl;

    public ProductDto toDto(ProductEntity e) {
        return toDto(e, false);
    }
    public ProductDto toDto(ProductEntity e, boolean hasHistory) {
        ProductDto dto = new ProductDto();
        dto.setId(e.getId());
        dto.setName(e.getName());
        dto.setPrice(e.getPrice());
        dto.setImg(e.getImg() == null ? null : uploadUrl + "/" + e.getImg()); // URL completa
        dto.setActive(e.isActive());
        dto.setHasHistory(hasHistory);
        return dto;
    }
}

```

Por que no devolver la entidad directamente: (1) expondrías el esquema interno y campos sensibles; (2) al serializar relaciones LAZY fuera de transaccion se producen errores o consultas extra; (3) el DTO permite adaptar la forma de los datos al frontend sin tocar la BD.

14. Excepciones y `GlobalExceptionHandler`

Cada modulo tiene su excepcion de negocio: `ProductException`, `PurchaseException`, `RoleException`, `UserException`, `PaymentException`, etc. Llevan un `HttpStatus` opcional para decidir 400/404/409.

Un unico `@RestControllerAdvice` centraliza la respuesta JSON de error, siempre con el mismo formato: `{ timestamp, status, code, message }`.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductException.class)
    public ResponseEntity<ErrorResponse> handleProduct(ProductException ex) {
        return buildErrorResponse(ex.getStatus(), "PRODUCT_ERROR", ex.getMessage());
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> handle403(AccessDeniedException ex) {
        return buildErrorResponse(HttpStatus.FORBIDDEN, "ACCESS_DENIED",
            "No tienes permisos para realizar esta acción");
    }

    @ExceptionHandler(Exception.class) // red de seguridad
    public ResponseEntity<ErrorResponse> handle500(Exception e) {
        log.error("Error interno del servidor", e); // el detalle solo al log
        return buildErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR,
            "INTERNAL_ERROR", "Ocurrió un error interno en el servidor");
    }
}
```

Manejadores incluidos: negocios (400/404/409), `AccessDenied` (403), `AuthenticationException` (401), validacion `MethodArgumentNotValidException` (400 con detalle por campo), tipo incorrecto en la URL (400), JSON malformado (400), recurso estatico inexistente/imagen huerfana (404 en vez de 500) y el generico (500).

Buenas practicas aplicadas: no filtrar stacktraces ni mensajes internos en el 500; loguear el detalle en el servidor; usar el codigo HTTP correcto (409 para conflictos de integridad, no 500).

15. Clases de `config/` y bootstraps

Clase	Para que sirve
<code>SecurityConfig</code>	Cadena de filtros, CORS, <code>PasswordEncoder</code> (BCrypt), rutas publicas.
<code>JwtUtil</code>	Generar y validar tokens HS256.
<code>JwtFilter</code>	Autenticar cada request con el token.
<code>security/SecurityAuthorityMapper</code>	Convertir rol + permisos en autoridades.
<code>WebConfig</code>	Servir <code>/api/uploads/**</code> desde la carpeta <code>app.upload-dir</code> .
<code>TerminalConfig</code>	<code>@ConfigurationProperties("payment.terminal")</code> : host, puerto, credenciales de la terminal de pagos.
<code>RoleBootstrap</code> , <code>AdminBootstrap</code> , <code>PermissionBootstrap</code> , <code>RolePermissionBootstrap</code>	Siembra inicial de roles/permisos/admin (seccion 9.5).

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    private final String uploadDir; // @Value("${app.upload-dir:./uploads}")

    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/api/uploads/**")
            .addResourceLocations("file:" + uploadDir + "/");
    }
}
```

`@ConfigurationProperties(prefix = "payment.terminal")` en `TerminalConfig` mapea de forma tipada el bloque YAML `payment.terminal.*` a un bean, evitando multiples `@Value`.

16. Jobs programados (`@Scheduled`)

`VentasApplication` lleva `@EnableScheduling`, lo que habilita los metodos `@Scheduled`. El proyecto tiene `PaymentMonitorJob`:

```
@Slf4j
@Component
@RequiredArgsConstructor
public class PaymentMonitorJob {
    private final PaymentRepository paymentRepository;
    private final PaymentService paymentService;

    @Scheduled(fixedDelay = 300000) // cada 5 minutos
    public void monitorPendingPayments() {
        LocalDateTime fiveMinutesAgo = LocalDateTime.now().minusMinutes(5);
        List<PaymentEntity> pendientes =
            paymentRepository.findByStatusAndPaymentDateBefore(PaymentStatus.PENDING, fiveMinutesAgo);
        for (PaymentEntity p : pendientes) {
            paymentService.getPaymentStatus(p.getTransactionId()); // consulta la terminal
            Thread.sleep(1000);
        }
    }
}
```

Problema real que resuelve: si la terminal aprueba el cobro pero el cliente pierde la conexión, el pago queda `PENDING`. El job lo "rescata" consultando la terminal periódicamente y actualizando la BD. `fixedDelay` espera a que termine una ejecución antes de contar el intervalo (a diferencia de `fixedRate`).

17. Tests: tipos, ejemplos y como ejecutarlos

La suite tiene **130 tests** (21 clases) y se ejecuta con `mvnw test`. Hay tres niveles:

a) Tests de repositorio / consultas: `@DataJpaTest` sobre H2

```
@DataJpaTest(properties = "spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect")
class ProductRepositoryTest {
    @Autowired ProductRepository productRepository;    // repositorio real, BD H2 en memoria

    @Test
    void search_esInsensibleAMayusculasYBuscaEnVariosCampos() {
        assertThat(productRepository.search("coca")).hasSize(1);
        assertThat(productRepository.search("pe-0")).hasSize(1);
    }
}
```

Ventaja: valida las consultas reales (JPQL/derived) sin necesitar Postgres. El dialogo H2 se fuerza con la propiedad `hibernate.dialect`.

b) Tests de controlador: `@WebMvcTest` + `MockMvc` + `@WithMockUser`

```
@WebMvcTest(ProductController.class)    // levanta solo la capa web
@Import(SecurityConfig.class)
class ProductControllerTest {
    @Autowired MockMvc mockMvc;
    @MockitoBean ProductService productService;    // el service se simula (mock)

    @Test
    @WithMockUser(authorities = "ELIMINAR_PRODUCTOS") // usuario fake con permiso
    void eliminarProducto_conHistorico_devuelve409() throws Exception {
        doThrow(new ProductException("...", HttpStatus.CONFLICT))
            .when(productService).delete(10L);

        mockMvc.perform(delete("/api/products/10"))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("PRODUCT_ERROR"));

        verify(productService).delete(10L);
    }

    @Test
    @WithMockUser(authorities = "VER_PRODUCTOS")
    void darDeBajaProducto_sinPermiso_devuelve403() throws Exception {
        mockMvc.perform(patch("/api/products/10/deactivate"))
            .andExpect(status().isForbidden());    // falta DESACTIVAR_PRODUCTOS
    }
}
```

c) Tests de service: Mockito puro (sin Spring)

```
class PurchaseImplTest {
    @Mock PurchaseRepository purchaseRepository;
    @Mock ProductRepository productRepository;
    @InjectMocks PurchaseImpl purchaseService; // constructor inyectado con los mocks

    @Test
    void create_sumaStockYGuardaCosto() {
        // given... when patear create... then verificar stock y cost del producto
    }
}
```

Anotacion / utilidad	Uso
@Test	Cada caso de prueba (JUnit 5).
@DataJpaTest	Configura JPA + BD en memoria y transacciona cada test (rollback).
@WebMvcTest	Carga solo controllers/serializacion/seguridad, no la BD.
@MockitoBean / @Mock / @InjectMocks	Objetos simulados para aislar la unidad bajo prueba.
@WithMockUser(authorities=...)	Simula un usuario autenticado con permisos, sin JWT real.
MockMvc	Ejecuta peticiones HTTP contra el controller y verifica estado/JSON.
assertThat (AssertJ)	Aserciones legibles (isPresent() , hasSize()).

Piramide de tests: muchos tests rapidos de servicio/repositorio, algunos de controller (capa web) y pocos de integracion completos. Aqui el unico test "de aplicacion" que levanta todo el contexto es `VentasApplicationTests (contextLoads)`.

18. Comandos utiles (Maven wrapper y Docker)

Comando	Que hace
<code>.\mvnw.cmd compile</code>	Compila.
<code>.\mvnw.cmd test</code>	Ejecuta los 130 tests (necesita BD para el test de contexto; los de H2 no).
<code>.\mvnw.cmd test -Dtest="ProductControllerTest"</code>	Ejecuta una sola clase de tests.
<code>.\mvnw.cmd clean package</code>	Empaqueta el jar ejecutable en <code>target/</code> .
<code>.\mvnw.cmd spring-boot:run</code>	Arranca la API en <code>http://localhost:8081</code> .
<code>docker build -t ventas-backend .</code>	Construye la imagen con el Dockerfile multi-etapa.
<code>docker run -p 8081:8081 --env-file .env ventas-backend</code>	Ejecuta pasando variables de entorno.

En Linux/macOS se usa `./mvnw`; en Windows `mvnw.cmd`. El wrapper garantiza la version de Maven sin instalacion previa.

19. Catalogo de endpoints por modulo

Modulo	Base	Ejemplos	Permiso principal
Auth	/api/auth	login, forgot/reset/change-password, me/authorities	publico / autenticado
Users	/api/users	CRUD + PATCH /{id}/active	VER/CREAR/EDITAR_USUARIOS
Roles	/api/roles	CRUD	VER/CREAR/EDITAR/ELIMINAR_ROLES
Permissions	/api/permissions	GET all	autenticado
Products	/api/products	CRUD multipart, /barcode/{bc} , /search?q= , /inactive , PATCH /{id}/deactivate active	VER/CREAR/EDITAR/ELIMINAR/DESACTIVAR/ACTIVAR_PRODUCTOS
Categories	/api/categories	CRUD	VER/CREAR/EDITAR/ELIMINAR_CATEGORIAS
Sales	/api/sales	POST (efectivo/tarjeta), GET historial, GET por id	VER/CREAR_VENTAS
Payments	/api/payments	/card , /status/{tx} , /retry/{id} , /reverse/{id}	PROCESAR_PAGOS
Cash	/api/cash	open, close, summary, active	VER/ABRIR/CERRAR/CORTE_CAJA
Purchases	/api/purchases	GET (todos/por proveedor/detalle), POST, DELETE (cancelar)	VER/CREAR/CANCELAR_COMPRAS
Providers	/api/providers	CRUD	VER/CREAR/EDITAR/ELIMINAR_PROVEEDORES
Reports	/api/reports	trend, top-products, payment-methods, categories, low-stock, margins, summary	VER_REPORTES
Uploads	/api/uploads/**	sirve imagenes de productos	publico

Total: aproximadamente **56 endpoints** distribuidos en **12 controllers**.

20. Flujos clave paso a paso

Login

1. El cliente envia email/contraseña a `POST /api/auth/login`.
2. `AuthImpl` busca el usuario con rol y permisos (`JOIN FETCH`), verifica que este activo y compara con `passwordEncoder.matches()` (BCrypt).
3. Si coincide, `JwtUtil` genera el token con `subject = email` y devuelve tambien rol y permisos.
4. Si no coincide, responde `success=false` (no revela si el email existe: evita enumeracion de usuarios).

Venta con tarjeta

1. Verifica que exista caja abierta.
2. Crea la venta en estado `PENDING` (para tener ID) y calcula el total validando stock.
3. Llama a `PaymentService`, que delega en la terminal (`TerminalService` simulada o fisica).
4. Si aprueba: marca `APPROVED` y descuenta stock. Si rechaza: excepcion => rollback.

Compra a proveedor

1. Valida proveedor e items. Calcula subtotales y total.
2. Por cada renglon: suma stock y guarda el costo real (`product.cost = unitCost`).
3. Cancelar una compra revierte el stock (nunca por debajo de 0) y elimina la compra; el costo se conserva.

Borrado logico de producto

1. Si el producto tiene ventas/compras, el borrado fisico responde `409`.
2. Alternativa: `PATCH /{id}/deactivate` pone `active=false` (sale del catalogo, conserva historico).
3. `PATCH /{id}/active` lo reactiva. El POS no muestra ni busca productos dados de baja.

Recuperacion de contraseña

1. `forgot-password` : genera un token UUID con 1 h de validez y envia un enlace `${app.frontend-url}/reset-password?token=...` por correo.
2. `reset-password` : valida el token y su expiracion, y guarda la nueva contraseña hasheada.

21. Decisiones de diseno y buenas practicas

- **DTOs en la frontera:** las entidades no cruzan HTTP.
- **Inyeccion por constructor:** dependencias explicitas e inmutables (`final`), facilitan tests.
- **Separar interfaz e implementacion de service:** desacople y testeabilidad.
- **Autorizacion por permiso** y no solo por rol: mas granular, y los permisos se recargan por request.
- **Secretos fuera del codigo:** `${VAR}` + `application-local.yaml` ignorado.
- **Fail-fast:** si falta una variable obligatoria, no arranca.
- **Transacciones** en operaciones multi-tabla (venta, compra) con `@Transactional`.
- **Evitar N+1** con `@EntityGraph` y `JOIN FETCH`.
- **Agregaciones en SQL** para reportes, no en memoria.
- **Manejo de errores uniforme** y codigos HTTP correctos (409 en conflictos).
- **Integridad del historico:** no borrar lo que tiene ventas/compras; borrado logico.
- **Imagenes** validadas (tamano, extension, contenido) y servidas aparte; huérfanas => 404.

22. Deuda tecnica y posibles mejoras

- **Migraciones reales:** usar Flyway/Liquibase en vez de `ddl-auto: update` (y `validate` en produccion).
- **El CHECK de permissions :** Hibernate genera `permissions_name_check` para el `@Enumerated(String)` y `ddl-auto: update` NO lo actualiza; al agregar permisos al enum, en una BD existente hay que dropear el constraint (documentado en AGENTS.md). Un `PermissionEntity.name` como `String` sin CHECK o una migracion explicita lo evitaria.
- **Logs:** cambiar `System.out.println` de los bootstraps por el logger (`@Slf4j`).
- **Tests de integracion end-to-end** con Testcontainers (Postgres real) para pagos y compras.
- **Paginacion** en listados grandes (ventas, productos) con `Pageable`.
- **Rate limiting / bloqueo por intentos** en login.
- **Refresh tokens** para no forzar re-login cada 5 horas.
- **Idempotencia explicita** en pagos (clave de idempotencia persistida).
- **Cobertura** (JaCoCo) y analisis estatico (Sonar).

Pendiente de seguridad importante: el historial de Git contiene secretos antiguos. Antes de publicar el repo hay que purgarlo con `git filter-repo`.

23. Preguntas típicas de entrevista (con respuestas del proyecto)

¿Que es Spring Boot y en que se diferencia de Spring?

Spring es el framework base (IoC, DI, MVC). Spring Boot lo simplifica con auto-configuracion, starters y servidor embebido: no necesitas XML ni desplegar un WAR en un Tomcat externo. Aqui basta `mvnw spring-boot:run`.

¿Que es la inyeccion de dependencias?

Spring crea y administra los beans; las clases declaran lo que necesitan y el contenedor lo entrega. En este proyecto se usa inyeccion por constructor: `public ProductController(ProductService service) { this.service = service; }`.

¿Diferencia entre @Component, @Service, @Repository y @Controller ?

Todos registran un bean. `@Service` (logica de negocio), `@Repository` (acceso a datos, traduce excepciones de persistencia) y `@Controller` (web) son especializaciones semanticas de `@Component`.

¿Como maneja Spring los errores?

Con `@RestControllerAdvice`: un metodo `@ExceptionHandler` por tipo de excepcion devuelve un JSON uniforme. Ver `GlobalExceptionHandler` (seccion 14).

¿Como funciona la seguridad con JWT?

El login emite un token firmado HS256; un filtro (`JwtFilter`) lo lee en cada request, valida la firma, carga al usuario/permisos y coloca la autentificacion en el `SecurityContext`. Luego `@PreAuthorize` autoriza por permiso. Es stateless: no hay sesion en servidor.

¿Por que no guardas los permisos en el token?

Por frescura: si quitas un permiso a un rol, queremos que aplique de inmediato sin esperar la expiracion. El token solo lleva el email; los permisos se recargan de BD cada request.

¿Que es una transaccion y cuando la usas?

Una unidad atomica de trabajo (ACID). Se usa `@Transactional` en operaciones que tocan varias tablas o recursos: crear/cancelar venta y compra. Si falla algo, hace rollback de todo.

¿Que es el problema N+1 y como lo evitas?

Cargar una lista y luego disparar una consulta por elemento (1 + N). Se evita con `@EntityGraph` o `JOIN FETCH` para precargar relaciones en una sola consulta (ver `PurchaseRepository`, `UserRepository`).

¿Por que usas BigDecimal para dinero?

Porque `double` / `float` tienen errores de redondeo binarios; `BigDecimal` es exacto y permite fijar escala/redondeo (`setScale(2, HALF_UP)`).

¿Como optimizas los reportes?

Agregando en la base de datos (SUM/COUNT/GROUP BY) y devolviendo solo el resultado; el frontend grafica. Se filtran ventas `APPROVED` por rango de fechas.

¿Que diferencia hay entre ddl-auto: update y validate ?

`update` modifica el esquema automaticamente (comodo en dev, riesgoso en prod). `validate` solo verifica que coincida; en produccion se usan migraciones versionadas (Flyway/Liquibase).

¿Como evitas que se borre informacion historica?

Bloqueando el borrado fisico si hay ventas/compras (409) y usando borrado logico (`active`) para productos y usuarios.

¿Que es @ConfigurationProperties ?

Una forma tipada de enlazar un bloque de configuracion YAML a un bean Java. Ver `TerminalConfig` con `@ConfigurationProperties(prefix="payment.terminal")` .

¿Como pruebas la capa web sin base de datos?

Con `@WebMvcTest` + `MockMvc`, simulando el service con `@MockitoBean` y el usuario con `@WithMockUser` . Ver `ProductControllerTest` .

¿Que es un CommandLineRunner ?

Codigo que se ejecuta al terminar de arrancar el contexto. Aqui siembra roles, admin y permisos (`RoleBootstrap`, `AdminBootstrap`, `PermissionBootstrap`, `RolePermissionBootstrap`).

¿Que es fixedDelay vs fixedRate en @Scheduled ?

`fixedDelay` mide desde que termina la ejecucion anterior; `fixedRate` desde que empieza. El job de pagos usa `fixedDelay` para no solapar ejecuciones.

¿Que es Docker multi-etapa y por que?

Varias instrucciones `FROM` en un `Dockerfile`. La primera compila (Maven+JDK) y la segunda solo ejecuta (JRE), copiando el jar. Imagen mas pequena y sin herramientas de build ni secretos.

24. Glosario rapido

Termino	Definicion breve
Bean	Objeto administrado por el contenedor de Spring.
Starter	Dependencia que agrupa librerias y auto-configuracion (p. ej. <code>starter-web</code>).
DTO	Objeto para transferir datos entre capas/API.
Entity	Clase mapeada a una tabla de BD.
Repository	Interfaz de acceso a datos; Spring Data genera la implementacion.
JPQL	Lenguaje de consulta sobre entidades (no tablas).
N+1	Problema de rendimiento: una consulta por elemento de una lista.
Lazy/Eager	Carga diferida o inmediata de una relacion.
JWT	Token firmado que transporta identidad (aqui el email).
BCrypt	Algoritmo de hash de contraseñas con salt.
Orphan removal	Borra la fila hija cuando se quita del padre.
Rollback	Deshacer una transaccion ante un error.
Soft delete / borrado logico	Marcar como inactivo sin borrar la fila (conserva historico).
Healthcheck	Endpoint usado por la plataforma para saber si la app sigue viva.